

Data Structures and Algorithms for Problem Solving (CS1.304)

Longest Common Subsequence (LCS)

Instructor: Kshitij Gajjar

Longest Common Subsequence (LCS)

Find the longest common subsequence between the following strings:

bafdecafdc
abcdef

Longest Common Subsequence (LCS)

Find the longest common subsequence between the following strings:

bafdecafdc
abcdef

We will analyse the algorithm for it while running through its dynamic program (DP) in a step-by-step manner.

Longest Common Subsequence (LCS)

We start by building a DP (dynamic program) table. The characters of the first string are the headers for the columns, and the characters of the second string are the headers for the rows.

[illegible]

Longest Common Subsequence (LCS)

The first row and the first column of the DP table are all zeroes.

[illegible]

Longest Common Subsequence (LCS)

Now let us start filling in the entries of this table. We will take a pause after some time to see how it is being done.

		b	a	f	d	e	c
	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1
b	0	1	1	1	1	1	1
c	0	1	1	1	1	1	X

Pause: Let us understand this DP. Suppose that we need to compute the cell marked as **X** in this table. For this, scan the strings from the start up to (and including) the cell that is marked as **X**. They are:

- **bafdec**
- **abc**

		b	a	f	d	e	c
	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1
b	0	1	1	1	1	1	1
c	0	1	1	1	1	1	X

Pause: Let us understand this DP. Suppose that we need to compute the cell marked as **X** in this table. For this, scan the strings from the start up to (and including) the cell that is marked as **X**. They are:

- **bafdec**
- **abc**

The ending characters (**c** and **c**) of these two strings is the same. And since we need to compute a **longest** subsequence that is **common** to both of the strings, **c** can extend that subsequence, as it occurs in both the strings.

		b	a	f	d	e	c
	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1
b	0	1	1	1	1	1	1
c	0	1	1	1	1	1	X

Pause: Let us understand this DP. Suppose that we need to compute the cell marked as **X** in this table. For this, scan the strings from the start up to (and including) the cell that is marked as **X**. They are:

- **bafdec**
- **abc**

So we need to know the LCS (longest common subsequence) of the strings:

- **bafdec**
- **abc**

Their LCS is already there in the cell **diagonally** before it!

		b	a	f	d	e	c
	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1
b	0	1	1	1	1	1	1
c	0	1	1	1	1	1	2

Pause: Let us understand this DP. Suppose that we need to compute the cell marked as **X** in this table. For this, scan the strings from the start up to (and including) the cell that is marked as **X**. They are:

- **bafdec**
- **abc**

So we need to know the LCS (longest common subsequence) of the strings:

- **bafdec**
- **abc**

Their LCS is already there in the cell **diagonally** before it! Just add 1 to it.

		b	a	f	d	e
	0	0	0	0	0	0
a	0 → 0	1	1	1	1	1
b	0	1	1	1	1	1
c	0	1	1	1	1	1
d	0	1	1	1	2	X

Pause: We need to compute the **X** cell here. Scan both of the strings from their start up to (and including) the **X** cell. They are:

- **bafde**
- **abcd**

		b	a	f	d	e
	0	0	0	0	0	0
a	0	0	1	1	1	1
b	0	1	1	1	1	1
c	0	1	1	1	1	1
d	0	1	1	1	2	X

Pause: We need to compute the **X** cell here. Scan both of the strings from their start up to (and including) the **X** cell. They are:

- **bafde**
- **abcd**

The ending characters (**e** and **d**) of these two strings are not the same. And since we need to compute a longest subsequence that is **common** to both the strings, **e** and **d** cannot be part of that subsequence simultaneously. So we have to leave one of them out: we can either include **e** or include **d**.

		b	a	f	d	e
	0	0	0	0	0	0
a	0	0	1	1	1	1
b	0	1	1	1	1	1
c	0	1	1	1	1	1
d	0	1	1	1	2	X

Pause: We need to compute the **X** cell here. Scan both of the strings from their start up to (and including) the **X** cell. They are:

- **bafde**
- **abcd**

If we include **e** and leave out **d**, then we need the LCS of the two strings:

- **bafde**
- **abcd**

And their LCS is already there in the cell **above** it!

		b	a	f	d	e
	0	0	0	0	0	0
a	0	0	1	1	1	1
b	0	1	1	1	1	1
c	0	1	1	1	1	1
d	0	1	1	1	2	X

Pause: We need to compute the **X** cell here. Scan both of the strings from their start up to (and including) the **X** cell. They are:

- **bafde**
- **abcd**

If we include **d** and leave out **e**, then we need the LCS of the two strings:

- **bafde**
- **abcd**

And their LCS is already there in the cell **beside** it!

		b	a	f	d	e
	0	0	0	0	0	0
a	0	0	1	1	1	1
b	0	1	1	1	1	1
c	0	1	1	1	1	1
d	0	1	1	1	2	X

Pause: We need to compute the **X** cell here. Scan both of the strings from their start up to (and including) the **X** cell. They are:

- **bafde**
- **abcd**

We need to compute a **longest** common subsequence (LCS), so we choose the **maximum** of the two numbers, and fill it up in our **X** cell.

		b	a	f	d	e
	0	0	0	0	0	0
a	0	0	1	1	1	1
b	0	1	1	1	1	1
c	0	1	1	1	1	1
d	0	1	1	1	2	2

Pause: We need to compute the **X** cell here. Scan both of the strings from their start up to (and including) the **X** cell. They are:

- **bafde**
- **abcd**

We need to compute a **longest** common subsequence (LCS), so we choose the **maximum** of the two numbers, and fill it up in our **X** cell.

The → arrow indicates where that number came from.

		b
	0	0
a	0	0
b	0	1
c	0	1
d	0	1
e	0	x

Pause: We need to compute the **X** cell here. Scan the strings from their start up to (and including) the **X** cell. They are:

- **b**
- **abcde**

		b
	0	0
a	0	0
b	0	1
c	0	1
d	0	1
e	0	X

Pause: We need to compute the **X** cell here. Scan the strings from their start up to (and including) the **X** cell. They are:

- **b**
- **abcde**

The ending characters (**b** and **e**) of these two strings are not the same. And since we need to compute a longest subsequence which is **common** to both the strings, **b** and **e** cannot be part of that subsequence simultaneously. Thus we need to leave one of them out: we can either include **b** or include **e**.

		b
	0	0
a	0	0
b	0	1
c	0	1
d	0	1
e	0	X

Pause: We need to compute the **X** cell here. Scan the strings from their start up to (and including) the **X** cell. They are:

- **b**
- **abcde**

If we include **b** and leave out **e**, then we need the LCS of the two strings:

- **b**
- **abcd****e**

And their LCS is already there in the cell **above** it.

		b
	0	0
a	0	0
b	0	1
c	0	1
d	0	1
e	0	x

Pause: We need to compute the **x** cell here. Scan the strings from their start up to (and including) the **x** cell. They are:

- **b**
- **abcde**

If we include **e** and leave out **b**, then we need the LCS of the two strings:

- **b**
- **abcde**

And their LCS is already there in the cell **beside** it.

		b
	0	0
a	0	0
b	0	1
c	0	1
d	0	1
e	0	X

Pause: We need to compute the **X** cell here. Scan the strings from their start up to (and including) the **X** cell. They are:

- **b**
- **abcde**

We need a **longest** common subsequence (LCS), so we choose the **maximum** of the two numbers, and fill it up in our **X** cell.

		b
	0	0
a	0	0
b	0	1
c	0	1
d	0	1
e	0	1

Pause: We need to compute the **X** cell here. Scan the strings from their start up to (and including) the **X** cell. They are:

- **b**
- **abcde**

We need a **longest** common subsequence (LCS), so we choose the **maximum** of the two numbers, and fill it up in our **X** cell.

The ↓ arrow denotes where the number came from.

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2
d	0	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2 → 2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 1	2 → 2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	1 → 1	2 → 2							

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1	1	1	1
b	0	1	1	1	1	1	1	1	1	1	1
c	0	1	1	1	1	1	2	2	2	2	2
d	0	1	1	1	2	2	2	2	2	3	3
e	0	1	1	1	2	3	3	3	3	3	3
f	0	1	1	2	2	3	3	3	4		

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2
d	0	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2 → 2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 1	2 → 2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	1 → 1	2 → 2	2 → 2	3 → 3	3 → 3	3 → 3	4 → 4	4 → 4	4

Longest Common Subsequence (LCS)

And we are done! The length of the LCS is the value present in the cell at the bottom right corner of the DP table.

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1	2 → 2	2 → 2	2 → 2	2	2
d	0	1 → 1	1 → 1	1	2 → 2	2 → 2	2 → 2	2 → 2	2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 2	2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	2 → 2	2 → 3	3 → 3	3	4 → 4	4 → 4	4	4	4

Longest Common Subsequence (LCS)

So, we know the **length** of the LCS. But how can we find the LCS? Now our arrows will help us. Recall that each cell had exactly one incoming arrow. We just follow these arrows backwards!

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2
d	0	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2 → 2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 1	2 → 2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	1 → 1	2 → 2	2 → 2	3 → 3	3 → 3	3 → 3	4 → 4	4 → 4	4

- 1
2
3
4

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1	1	1	1
b	0	1	1	1	1	1	1	1	1	1	1
c	0	1	1	1	1	1	2	2	2	2	2
d	0	1	1	1	2	2	2	2	2	3	3
e	0	1	1	1	2	3	3	3	3	3	3
f	0	1	1	2	2	3	3	3	4	4	4

1	2	3	4
---	---	---	---

Longest Common Subsequence (LCS)

Each time we encounter a diagonal  arrow incoming to the cell that we are currently at, we include that character in our LCS.

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2
d	0	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2 → 2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 1	2 → 2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	1 → 1	2 → 2	2 → 2	3 → 3	3 → 3	3 → 3	4 → 4	4 → 4	4

1
2
3
4

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1	1	1	1
b	0	1	1	1	1	1	1	1	1	1	1
c	0	1	1	1	1	1	2	2	2	2	2
d	0	1	1	1	2	2	2	2	2	3	3
e	0	1	1	1	2	3	3	3	3	3	3
f	0	1	1	2	2	3	3	3	4	4	4

			f
1	2	3	4

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2
d	0	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2 → 2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 1	2 → 2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	1 → 1	2 → 2	2 → 2	3 → 3	3 → 3	3 → 3	4 → 4	4 → 4	4

			f
1	2	3	4

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1	1	1	1
b	0	1	1	1	1	1	1	1	1	1	1
c	0	1	1	1	1	1	2	2	2	2	2
d	0	1	1	1	2	2	2	2	2	3	3
e	0	1	1	1	2	3	3	3	3	3	3
f	0	1	1	2	2	3	3	3	4	4	4

		e	f
1	2	3	4

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2
d	0	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2 → 2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 1	2 → 2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	1 → 1	2 → 2	2 → 2	3 → 3	3 → 3	3 → 3	4 → 4	4 → 4	4

	d	e	f
1	2	3	4

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0 → 0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
b	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1
c	0	1 → 1	1 → 1	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2
d	0	1 → 1	1 → 1	1 → 1	2 → 2	2 → 2	2 → 2	2 → 2	2 → 2	3 → 3	3
e	0	1 → 1	1 → 1	1 → 1	2 → 2	3 → 3	3 → 3	3 → 3	3 → 3	3 → 3	3
f	0	1 → 1	1 → 1	2 → 2	2 → 2	3 → 3	3 → 3	3 → 3	4 → 4	4 → 4	4

d	e	f
1	2	3
2	3	4

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1	1	1	1
b	0	1	1	1	1	1	1	1	1	1	1
c	0	1	1	1	1	1	2	2	2	2	2
d	0	1	1	1	2	2	2	2	2	3	3
e	0	1	1	1	2	3	3	3	3	3	3
f	0	1	1	2	2	3	3	3	4	4	4

d	e	f
1	2	3
4		

		b	a	f	d	e	c	a	f	d	c
	0	0	0	0	0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1	1	1	1
b	0	1	1	1	1	1	1	1	1	1	1
c	0	1	1	1	1	1	2	2	2	2	2
d	0	1	1	1	2	2	2	2	2	3	3
e	0	1	1	1	2	3	3	3	3	3	3
f	0	1	1	2	2	3	3	3	4	4	4

b	d	e	f
1	2	3	4

Longest Common Subsequence (LCS)

Hence, the longest common subsequence (LCS) between the strings **bafdecafdc** and **abcdef** is:

bdef

Thank You!

